

Dan Boneh 《Cryptography I》 第二课：流密码

依据 Stanford Online Cryptography Course 原版讲义 [02-stream-v2-annotated.pdf](#) (83 页) 整理。
本讲主线：一次一密 → 伪随机生成器 → 流密码 → 典型攻击 → PRG 安全性 → 语义安全 → 归约证明。

0. 本课要解决的问题

一次一密 (One-Time Pad, OTP) 具有完美保密性，但密钥必须与消息一样长，实际使用极不方便。

流密码的核心想法是：

不再真正随机地产生一条与消息等长的密钥，而是用一个较短的随机种子，通过伪随机生成器扩展出一条很长、看起来像随机的密钥流。

整个逻辑链如下：

```
短随机密钥 k
    ↓
伪随机生成器 G
    ↓
长密钥流 G(k)
    ↓ 与明文 m 异或
密文 c = m XOR G(k)
```

学完本课，应能回答：

1. OTP 为什么安全，又为什么难以实用？
2. 流密码为什么不可能具有完美保密性？
3. “伪随机”究竟是什么意思？
4. 为什么同一密钥流绝不能使用两次？
5. 为什么保密并不等于完整性？
6. 如何用“计算不可区分性”定义 PRG 安全？
7. 为什么安全 PRG 能导出语义安全的流密码？

1. 对称密码的基本定义

一个定义在密钥空间 $\mathcal{K}$ 、消息空间 $\mathcal{M}$ 、密文空间 $\mathcal{C}$ 上的密码系统，由两个高效算法组成：

$$E : \mathcal{K} \times \mathcal{M} \rightarrow \mathcal{C}$$

$$D : \mathcal{K} \times \mathcal{C} \rightarrow \mathcal{M}$$

- E ：加密算法，可能是随机化算法。
- D ：解密算法，通常是确定性算法。
- 正确性要求：

$$\forall k \in \mathcal{K}, \forall m \in \mathcal{M}, D(k, E(k, m)) = m$$

这里的“高效”通常指算法运行时间是安全参数的多项式。

1.1 正确性不等于安全性

一个系统能正确加密和解密，只说明它“可用”，并不说明攻击者不能从密文中获得信息。

例如替换密码、维吉尼亚密码都满足正确性，但会泄露语言统计特征，因此并不安全。

2. 一次一密 OTP

2.1 定义

设消息是 n 位比特串：

$$\mathcal{M} = \mathcal{K} = \mathcal{C} = \{0, 1\}^n$$

密钥 k 从 $\{0, 1\}^n$ 中均匀随机选取，并且只使用一次。

加密：

$$E(k, m) = m \oplus k$$

解密：

$$D(k, c) = c \oplus k$$

因为异或满足 $x \oplus x = 0$ 、 $x \oplus 0 = x$ ，所以：

$$D(k, E(k, m)) = (m \oplus k) \oplus k = m$$

2.2 小例子

```
明文 m: 0 1 1 0 1 1 1
密钥 k: 1 0 1 1 0 1 0
----- XOR
密文 c: 1 1 0 1 1 0 1
```

已知明文和密文时，可以恢复本次使用的密钥：

$$k = m \oplus c$$

这并不违背 OTP 的安全性。密码学并不要求攻击者在得到一组明密文后仍无法求出对应的一次性密钥；关键是该密钥已经不能再被使用。

2.3 OTP 的优缺点

优点：

- 加解密极快，只需异或。

- 在正确使用时具有信息论意义上的完美保密性。

缺点：

- 密钥必须与消息一样长。
- 密钥必须真正均匀随机。
- 密钥必须预先安全分发。
- 每个密钥只能使用一次。

3. 完美保密性

3.1 为什么“无法恢复密钥”不是合适的安全定义

以下要求都太弱：

- 攻击者不能恢复密钥。
- 攻击者不能恢复完整明文。

即使攻击者无法恢复完整明文，但能够知道明文的某一位、消息主题或金额范围，系统仍然泄密。

Shannon 的思想是：

密文不应泄露关于明文的任何信息。

3.2 完美保密的定义

密码系统 (E, D) 具有完美保密性，如果对任意两个等长消息 m_0, m_1 和任意密文 c ，都有：

$$\Pr_{k \leftarrow \mathcal{K}}[E(k, m_0) = c] = \Pr_{k \leftarrow \mathcal{K}}[E(k, m_1) = c]$$

直观理解：观察到同一个密文 c 后，攻击者无法判断被加密的是 m_0 还是 m_1 。

必须要求 $|m_0| = |m_1|$ ，因为普通加密通常会泄露消息长度。

3.3 OTP 为什么具有完美保密性

固定消息 m 和密文 c 。要使

$$m \oplus k = c$$

成立，密钥只能是：

$$k = m \oplus c$$

在 2^n 个均匀随机密钥中，恰好只有一个密钥满足条件，因此：

$$\Pr[E(k, m) = c] = \frac{1}{2^n}$$

这个概率与 m 的具体内容无关，所以对于任意 m_0, m_1 ：

$$\Pr[E(k, m_0) = c] = \Pr[E(k, m_1) = c] = \frac{1}{2^n}$$

故 OTP 具有完美保密性。

3.4 坏消息：密钥长度下界

任何具有完美保密性的密码系统都必须满足：

$$|\mathcal{K}| \geq |\mathcal{M}|$$

对定长比特串而言，这意味着：

$$\text{key length} \geq \text{message length}$$

因此，若想使用短密钥加密长消息，就必须放弃无条件的完美保密，转而追求计算安全性。

4. 用 PRG 让 OTP 变得实用

4.1 伪随机生成器

伪随机生成器 (Pseudorandom Generator, PRG) 将短种子扩展成长比特串：

$$G : \{0, 1\}^s \rightarrow \{0, 1\}^n, \quad n \gg s$$

- 输入 k ：短而真正随机的种子，也作为流密码密钥。
- 输出 $G(k)$ ：长伪随机串，即密钥流。

4.2 流密码构造

加密：

$$E(k, m) = m \oplus G(k)$$

解密：

$$D(k, c) = c \oplus G(k)$$

正确性：

$$D(k, E(k, m)) = (m \oplus G(k)) \oplus G(k) = m$$

4.3 流密码为何没有完美保密性

若种子只有 s 位，而输出有 n 位，且 $s < n$ ，则 PRG 最多只能产生 2^s 种输出，但所有 n 位串共有 2^n 种。

因此：

$$|\text{Im}(G)| \leq 2^s < 2^n$$

绝大多数 n 位串根本不可能成为 $G(k)$ 。只要攻击者拥有无限计算能力，就可以枚举所有种子并区分 PRG 输出和真正随机串。

所以流密码只能达到：

对现实中计算能力有限的攻击者而言，密钥流与真正随机串不可区分。

5. PRG 的不可预测性

5.1 下一位不可预测

设：

$$G(k) = b_1 b_2 \cdots b_n$$

如果存在高效算法 A ，在看到前 i 位后，可以以明显高于随机猜测的概率预测第 $i + 1$ 位：

$$\Pr[A(b_1, \dots, b_i) = b_{i+1}] > \frac{1}{2} + \varepsilon$$

其中 ε 是不可忽略量，则称 G 在位置 i 可预测。

若所有位置都不存在这样的高效预测器，则 PRG 不可预测。

5.2 一个容易忽略的例子

假设所有输出都满足：

$$b_1 \oplus b_2 \oplus \cdots \oplus b_n = 1$$

那么在知道前 $n - 1$ 位后，可以精确算出最后一位：

$$b_n = 1 \oplus b_1 \oplus \cdots \oplus b_{n-1}$$

因此该生成器是可预测的，不能用于密码学。

5.3 普通随机数生成器不等于密码学 PRG

课程以旧版 `glibc random()` 为例：其内部状态按线性递推更新。此类生成器适合模拟、抽样，但输出之间具有可利用的代数关系，不适合生成密钥流。

密码学 PRG 需要满足：即使攻击者知道算法并观察到大量输出，也不能有效预测未知输出或恢复内部状态。

6. 可忽略量与不可忽略量

6.1 实际视角

课程给出的直觉尺度：

- $\varepsilon \geq 2^{-30}$ ：可能在处理约 1 GB 数据时发生，不能忽略。
- $\varepsilon \leq 2^{-80}$ ：在单个密钥的生命周期内极难发生，可视为可忽略。

这些数字用于建立直觉，不是永久固定的工程标准。

6.2 渐近定义

令 λ 为安全参数， $\varepsilon(\lambda)$ 为攻击优势。

若对任意常数 $d > 0$ ，当 λ 充分大时都有：

$$\varepsilon(\lambda) \leq \frac{1}{\lambda^d}$$

则称 ε 是可忽略函数。

换言之，可忽略函数比任意多项式倒数下降得都快。

例子：

- $2^{-\lambda}$ ：可忽略。
- λ^{-1000} ：不可忽略，因为它本身就是多项式倒数。
- 仅在奇数 λ 时取 $2^{-\lambda}$ 、偶数时取 λ^{-1000} ：不可忽略。

6.3 参数化 PRG

严格来说，PRG 是一族算法：

$$G_\lambda : \mathcal{K}_\lambda \rightarrow \{0, 1\}^{n(\lambda)}$$

随着 λ 增大：

- 种子长度增加；
- 输出长度增加；
- 攻击成本增加；
- 成功攻击的优势应趋于可忽略。

7. 流密码最重要的攻击：密钥流复用

7.1 Two-Time Pad

若用同一个密钥流加密两条消息：

$$c_1 = m_1 \oplus G(k)$$

$$c_2 = m_2 \oplus G(k)$$

攻击者把两个密文异或：

$$c_1 \oplus c_2 = (m_1 \oplus G(k)) \oplus (m_2 \oplus G(k)) = m_1 \oplus m_2$$

密钥流被完全抵消。

攻击者虽然暂时没有直接得到 m_1 和 m_2 ，但自然语言、文件格式和网络协议中存在大量冗余，因此通常能逐步恢复两条消息。

7.2 ASCII 空格技巧

英文 ASCII 中：

```
' ' = 0x20
```

空格与英文字母异或，常会改变字母的大小写位。例如：

```
'A' XOR 0x20 = 'a'  
'a' XOR 0x20 = 'A'
```

若 $m_1 \oplus m_2$ 的某个位置像英文字母，则其中一条消息在该位置很可能是空格。攻击者可据此猜词并扩展已知明文。

7.3 Crib dragging

攻击者猜测其中一条消息含有常见片段 w ，计算：

$$(m_1 \oplus m_2) \oplus w$$

若结果在另一条消息中呈现合理文本，猜测就很可能正确。随后可继续猜测相邻内容。

7.4 现实案例

课程提到：

- **Project VENONA**：苏联通信中重复使用一次一密密钥页，使部分消息能够被恢复。
- **MS-PPTP**：通信双向使用相关密钥，导致安全问题；客户端到服务器和服务器到客户端应使用不同密钥。
- **WEP**：只有 24 位 IV，约 2^{24} 帧后必然重复，实际上受生日效应影响会更早发生碰撞；有些网卡重启后还会将 IV 归零。

7.5 规则

绝不能让同一密钥流加密两条不同消息。

更准确地说，现代带 nonce 的流密码要求同一密钥下的 $(key, nonce)$ 组合不能重复。

应用建议：

- 网络通信：每个会话协商新密钥，或严格管理唯一 nonce。
- 双向通信：区分两个方向的密钥或 nonce 空间。
- 磁盘加密：同一扇区会反复改写，通常采用专门的磁盘加密模式，而不是朴素流密码。

8. 第二类攻击：没有完整性

8.1 OTP 和流密码具有可篡改性

原密文：

$$c = m \oplus G(k)$$

攻击者选择一个掩码 p ，发送：

$$c' = c \oplus p$$

接收者解密：

$$c' \oplus G(k) = (m \oplus G(k) \oplus p) \oplus G(k) = m \oplus p$$

攻击者即使不知道明文和密钥，也能让明文发生可预测的位翻转。

8.2 关键结论

流密码本身只解决保密性，不自动提供完整性或身份认证。

- **保密性**：攻击者看不懂消息。
- **完整性**：攻击者不能在不被发现的情况下修改消息。
- **认证性**：接收者能确认消息来自合法发送者。

现代系统应使用认证加密（AE/AEAD），例如 AES-GCM 或 ChaCha20-Poly1305，而不是只做 `plaintext XOR keystream`。

9. 现实中的流密码

9.1 RC4：历史算法，不应再使用

RC4 曾广泛用于 HTTPS 和 WEP。课程指出其初始输出存在统计偏差，例如第二个输出字节为 0 的概率约为：

$$\Pr[Z_2 = 0] = \frac{2}{256}$$

而真正均匀随机时应为 $1/256$ 。单次偏差很小，但收集大量样本后可以累积成有效攻击。

此外 RC4 还存在相关密钥攻击等问题。结论：RC4 已淘汰，不应在新系统中使用。

9.2 LFSR 与 CSS

线性反馈移位寄存器（LFSR）硬件实现简单、速度快，但线性结构使其容易受到代数分析。

课程中的 CSS 使用两个 LFSR：

- 一个 17 位 LFSR；
- 一个 25 位 LFSR；
- 总种子只有 40 位。

攻击者枚举 2^{17} 个较短 LFSR 的状态，利用已知的文件前缀推导另一个 LFSR 的候选输出，再检验一致性，攻击复杂度约为 2^{17} ，远低于穷举 2^{40} 。

这说明：把几个弱线性组件组合起来，并不会自动得到强密码。

9.3 带 nonce 的现代结构

现代流密码通常写成：

$$\text{PRG} : \{0, 1\}^s \times \mathcal{R} \rightarrow \{0, 1\}^n$$

其中 $r \in \mathcal{R}$ 是 nonce：在固定密钥下不能重复的值。

$$E(k, m; r) = m \oplus \text{PRG}(k; r)$$

nonce:

- 通常不需要保密，可以与密文一起发送。
- 核心要求是同一密钥下不重复。
- 可以由计数器、协议序号或足够大的随机值产生，具体取决于算法规范。

9.4 Salsa20

课程以 eSTREAM 计划中的 Salsa20 为现代流密码示例。其思想是按块产生密钥流：

$$\text{Salsa20}(k; r) = H(k, (r, 0)) \parallel H(k, (r, 1)) \parallel \dots$$

其中计数器 i 保证同一 nonce 下的各个输出块不同。

课程强调：我们通常无法证明一个实际 PRG 在绝对意义上安全；实际信心来自公开分析后仍没有发现优于穷举的攻击。

补充：今天工程上更常见的是 Salsa20 的后继 ChaCha20，并通常与 Poly1305 组合成认证加密方案。

9.5 生成随机密钥与 nonce

系统级密码学随机数生成器会持续将外部熵混入内部状态。熵源可能包括：

- 硬件随机数指令；
- 设备中断时序；
- 操作系统收集的其他不可预测事件。

应用程序应调用操作系统提供的密码学安全随机接口，不应使用普通的模拟随机函数自行产生密钥。

10. PRG 的严格安全定义

10.1 统计测试

统计测试是一个高效算法：

$$A : \{0, 1\}^n \rightarrow \{0, 1\}$$

它试图判断输入来自：

1. PRG 输出 $G(k)$ ，其中 $k \leftarrow \mathcal{K}$ ；
2. 真正均匀随机串 $r \leftarrow \{0, 1\}^n$ 。

测试例子：

- 最高位是否为 1；
- 1 的个数是否显著偏离 $n/2$ ；
- 输出是否满足某个代数关系；
- 是否能被某个压缩算法异常压缩。

10.2 区分优势

定义测试 A 对 PRG G 的区分优势：

$$\text{Adv}_{\text{PRG}}[A, G] = |\Pr[A(G(k)) = 1] - \Pr[A(r) = 1]|$$

其中 k 和 r 分别均匀随机选取。

例如，若 $G(k)$ 的最高位以 $2/3$ 的概率为 1，而均匀随机串的最高位以 $1/2$ 的概率为 1，取测试：

$$A(x) = \text{MSB}(x)$$

则：

$$\text{Adv}_{\text{PRG}}[A, G] = \left| \frac{2}{3} - \frac{1}{2} \right| = \frac{1}{6}$$

这是明显的不可忽略优势，所以该 PRG 不安全。

10.3 安全 PRG

若对所有高效统计测试 A ，都有：

$$\text{Adv}_{\text{PRG}}[A, G]$$

为可忽略量，则称 G 是安全 PRG。

等价地：

$$\{G(k) : k \leftarrow \mathcal{K}\} \approx_p U_n$$

其中 $\approx_p$ 表示计算不可区分， U_n 表示 n 位均匀分布。

10.4 Yao 定理

课程给出的结论：

PRG 安全，当且仅当其输出的下一位不可被高效预测。

一个方向容易理解：若能预测下一位，就可构造区分器。

反方向是 Yao 定理的核心：如果某个高效算法能够区分 PRG 输出和真随机串，那么可从该区分器构造出某一位上的下一位预测器。

因此，“通过所有高效统计测试”与“每一位都不能根据前缀预测”在计算安全意义上等价。

11. 语义安全

11.1 安全游戏

为了定义“密文不泄露明文信息”，考虑如下实验：

1. 攻击者 A 选择两个等长消息 m_0, m_1 。
2. 挑战者随机选择 $b \leftarrow \{0, 1\}$ 。
3. 挑战者随机生成密钥 k ，返回：

$$c \leftarrow E(k, m_b)$$

4. 攻击者观察 c ，输出一个比特。

令 W_b 表示在实验 b 中攻击者输出 1 的事件，则课程定义的语义安全优势为：

$$\text{Adv}_{\text{SS}}[A, E] = |\Pr[W_0] - \Pr[W_1]|$$

若对所有高效攻击者 A ，该优势都是可忽略的，则 E 是语义安全的。

11.2 为什么由攻击者选择消息

攻击者必须明确给出 m_0, m_1 ，这样定义才能覆盖它最想区分、最容易利用的消息对。

如果系统泄露明文最低位，攻击者可选：

$$\text{LSB}(m_0) = 0, \quad \text{LSB}(m_1) = 1$$

然后从密文推断最低位，获得优势 1。因此系统不是语义安全的。

11.3 等长条件的重要性

若 $|m_0| \neq |m_1|$ ，攻击者可能只看密文长度就知道加密了哪条消息。因此标准定义要求挑战消息等长。

11.4 OTP 的语义安全性

对于任何固定消息 m ：

$$k \oplus m$$

在 k 均匀随机时也是均匀分布。因此 m_0 和 m_1 的密文分布完全相同，任意攻击者优势均为 0。

完美保密是信息论性质；语义安全是计算性质。完美保密蕴含语义安全，但反过来不成立。

12. 核心定理：安全 PRG 导出安全流密码

12.1 定理

若：

$$G : \mathcal{K} \rightarrow \{0, 1\}^n$$

是安全 PRG，则由它构造的流密码：

$$E(k, m) = m \oplus G(k)$$

在一次性密钥模型下是语义安全的。

更具体地，对任意语义安全攻击者 A ，可构造 PRG 区分器 B ，使：

$$\text{Adv}_{\text{SS}}[A, E] \leq 2 \text{Adv}_{\text{PRG}}[B, G]$$

12.2 混合实验直觉

真实实验中，挑战密文分别为：

$$m_0 \oplus G(k) \quad \text{和} \quad m_1 \oplus G(k)$$

因为 $G(k)$ 与真随机串 r 计算不可区分，所以可分别将它替换为 r ：

$$m_0 \oplus G(k) \approx_p m_0 \oplus r$$

$$m_1 \oplus G(k) \approx_p m_1 \oplus r$$

而 r 是均匀随机串，所以：

$$m_0 \oplus r$$

与

$$m_1 \oplus r$$

分布完全相同。这正是 OTP 的完美保密性。

因此：

$$m_0 \oplus G(k) \approx_p m_0 \oplus r =_d m_1 \oplus r \approx_p m_1 \oplus G(k)$$

其中 $=_d$ 表示分布完全相同。

12.3 归约证明的反证思路

假设存在高效攻击者 A 能以不可忽略优势区分流密码对 m_0, m_1 的加密。

构造区分器 B ：

1. B 收到一个挑战串 y ，它要么是 $G(k)$ ，要么是真随机串 r 。
2. B 运行 A ，取得 A 选择的 m_0, m_1 。
3. B 把 $c = m_0 \oplus y$ 交给 A 。
4. B 根据 A 的输出判断 y 来自 PRG 还是真随机分布。

若 $y = G(k)$ ， A 看到的是真实流密码密文；若 $y = r$ ， A 看到的是 OTP 密文。只要 A 能显著区分这些实验， B 就能显著区分 PRG 输出与真随机串，违背 G 的安全性。

12.4 因子 2 从哪里来

证明经过两个替换：

1. 将 $m_0 \oplus G(k)$ 替换为 $m_0 \oplus r$ ；
2. 将 $m_1 \oplus r$ 替换为 $m_1 \oplus G(k)$ 。

每次替换造成的差异最多为一个 PRG 区分优势。利用三角不等式，两项相加得到上界中的因子 2。

13. 本课概念对照

概念	攻击者能力	安全要求	是否无条件安全
完美保密	计算能力无限	不同明文的密文分布完全相同	是
PRG 安全	仅限高效算法	PRG 输出与真随机串计算不可区分	否
语义安全	仅限高效算法	无法有效区分两个自选等长明文的密文	否
完整性	可主动篡改密文	任何未经授权修改都应被发现	与保密性不同

14. 常见误区

误区 1：攻击者求不出密钥，所以系统安全

错误。攻击者可能不用恢复密钥，就能推断明文的一部分、区分消息或篡改内容。

误区 2：PRG 输出看起来杂乱，就是安全的

错误。视觉上随机、通过若干简单统计测试远远不够。安全要求它通过所有高效区分器的测试。

误区 3：nonce 是秘密值

通常错误。nonce 一般公开，关键是在同一密钥下不能重复。

误区 4：换一个 IV 就一定安全

错误。IV/nonce 的长度、生成方式以及它与密钥结合的方法都必须符合算法规范。WEA 的短 IV 和相关密钥结构就是反例。

误区 5：密文无法阅读，所以无法篡改

错误。流密码具有可篡改性，攻击者可以通过翻转密文位来可预测地翻转明文位。

误区 6：同一密钥下只要明文不同，就可以重复使用

错误。真正不能重复的是密钥流。若 (key, nonce) 相同，就会产生相同密钥流并形成 two-time pad。

误区 7：证明安全等于永远不会被攻破

错误。归约证明是在明确假设和攻击模型下成立的。实现漏洞、nonce 复用、侧信道或错误的随机数源仍可能破坏系统。

15. 复习用推导清单

考试或做题时应能独立写出：

1. OTP 正确性：

$$(m \oplus k) \oplus k = m$$

2. OTP 完美保密概率：

$$\Pr[E(k, m) = c] = 2^{-n}$$

3. Two-time pad 消去密钥流:

$$c_1 \oplus c_2 = m_1 \oplus m_2$$

4. 密文篡改效果:

$$D(k, c \oplus p) = m \oplus p$$

5. PRG 区分优势:

$$|\Pr[A(G(k)) = 1] - \Pr[A(r) = 1]|$$

6. 语义安全优势:

$$|\Pr[W_0] - \Pr[W_1]|$$

7. 安全 PRG 到流密码的混合链:

$$m_0 \oplus G(k) \approx_p m_0 \oplus r =_d m_1 \oplus r \approx_p m_1 \oplus G(k)$$

16. 自测题

题 1

为什么 OTP 的密钥必须真正随机，而不能是重复模式或普通伪随机函数的输出？

题 2

已知:

$$c_1 = m_1 \oplus z, \quad c_2 = m_2 \oplus z$$

攻击者能立即计算出什么？为什么这通常足以破坏安全性？

题 3

若一个 PRG 的所有输出最后一位都为 0，它是否安全？请构造区分器并计算优势。

题 4

为什么流密码的 nonce 可以公开，但不能重复？

题 5

攻击者把流密码密文 c 改为 $c \oplus p$ ，解密结果是什么？这说明系统缺少什么性质？

题 6

为什么语义安全游戏要求 m_0 和 m_1 等长？

题 7

安全 PRG 导出语义安全流密码的证明中，为什么要引入真正随机串 r ？

题 8

函数 $\varepsilon(\lambda) = 1/\lambda^{1000}$ 是否可忽略？为什么？

17. 自测题参考答案

答 1

若密钥存在可识别结构，密文就可能保留或泄露明文信息。真正的 OTP 要求密钥在所有 n 位串中均匀随机选择。用安全 PRG 扩展短种子得到的是流密码，只能提供计算安全性，不再是完美保密。

答 2

攻击者可计算：

$$c_1 \oplus c_2 = m_1 \oplus m_2$$

自然语言和文件格式有冗余，攻击者可利用空格、常见词、文件头或已知协议字段逐步恢复两条明文。

答 3

不安全。区分器读取最后一位：若为 0 输出 1，否则输出 0。对 PRG 输出，输出 1 的概率是 1；对真随机串，概率是 $1/2$ ，优势为 $1/2$ 。

答 4

nonce 本身不承担保密任务，它用于保证每次产生不同密钥流。若同一密钥下 nonce 重复， $\text{PRG}(k, \text{nonce})$ 也重复，从而形成 two-time pad。

答 5

解密结果是 $m \oplus p$ 。这说明攻击者可以可预测地改变明文，系统缺少完整性和认证性。

答 6

否则攻击者可以直接根据密文长度判断加密的是哪条消息，这种区分与密码算法本身无关。

答 7

真正随机串把流密码暂时变成 OTP。两条不同消息与均匀随机串异或后的分布完全相同，因此可把问题归约为“能否区分 $G(k)$ 与 r ”。

答 8

不可忽略。可忽略函数必须最终小于任意多项式倒数；取 $d = 1001$ 时， $1/\lambda^{1000}$ 并不小于 $1/\lambda^{1001}$ 。

18. 一页式总结

OTP

$$c = m \text{ XOR } k$$

优点：完美保密

缺点： $|k| \geq |m|$ ，且 k 只能用一次

流密码

$$c = m \text{ XOR } G(k)$$

用短随机种子生成伪随机密钥流

不具备完美保密，只追求计算安全

安全 PRG

任意高效区分器都无法把 $G(k)$ 与均匀随机串区分开

等价直觉：无法根据前缀预测下一位

最严重的使用错误

重复密钥流： $c_1 \text{ XOR } c_2 = m_1 \text{ XOR } m_2$

篡改密文： $D(k, c \text{ XOR } p) = m \text{ XOR } p$

现代实践

同一密钥下 `nonce` 不得重复

不使用 RC4、CSS 或普通随机数生成器

使用认证加密，如 ChaCha20-Poly1305 / AES-GCM

核心定理

安全 PRG + 一次性密钥模型

=> 流密码语义安全

19. 学习建议

第一次复习：掌握 OTP、流密码和 two-time pad 的四个异或推导。

第二次复习：理解可忽略量、统计测试、区分优势。

第三次复习：不看笔记，独立讲清“安全 PRG 为什么导出语义安全流密码”。

最后用第 16 节自测题检查薄弱点。

课程语境说明：讲义以一次性密钥模型引入语义安全。后续课程还会讨论允许攻击者获得多条密文、选择明文的更强安全模型，以及认证加密。